



Mono2Micro: An AI-Based Toolchain for Evolving Monolithic Enterprise Applications to a Microservice Architecture

Anup K. Kalia
Jin Xiao
anup.kalia@ibm.com
jinoaix@us.ibm.com
IBM Research
Yorktown Heights, NY
USA

Chen Lin
Saurabh Sinha
liana.lin@ibm.com
sinhas@us.ibm.com
IBM Research
Yorktown Heights, NY
USA

John Rofrano
Maja Vukovic
rofrano@us.ibm.com
maja@us.ibm.com
IBM Research
Yorktown Heights, NY
USA

Debasish Banerjee
debasish@us.ibm.com
IBM Hybrid Cloud
Rochester, MN, USA

ABSTRACT

Mono2Micro is an AI-based toolchain that provides recommendations for decomposing legacy web applications into microservice partitions. Mono2Micro consists of a set of tools that collect static and runtime information from a monolithic application and process the information using an AI-based technique to generate recommendations for partitioning the application classes. Each partition represents a candidate microservice or a grouping of classes with similar business functionalities. Mono2Micro takes a temporal-spatial clustering approach to compute meaningful and explainable partitions. It generates two types of partition recommendations. First, it computes business-logic-seams-based partitions that represent a desired encapsulation of business functionalities. However, such a recommendation may cut across data dependencies between classes, accommodating which could require significant application updates. To address this, Mono2Micro computes natural-seams-based partitions, which respect data dependencies. We describe the set of tools that comprise Mono2Micro and illustrate them using a well-known open-source JEE application.

CCS CONCEPTS

• **Software and its engineering** → **Software architectures**; *Software post-development issues*.

KEYWORDS

software modularization, microservices, dynamic analysis, clustering

ACM Reference Format:

Anup K. Kalia, Jin Xiao, Chen Lin, Saurabh Sinha, John Rofrano, Maja Vukovic, and Debasish Banerjee. 2020. Mono2Micro: An AI-Based Toolchain for Evolving Monolithic Enterprise Applications to a Microservice Architecture. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*, November 8–13, 2020, Virtual Event, USA. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3368089.3417933>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ESEC/FSE '20, November 8–13, 2020, Virtual Event, USA

© 2020 Association for Computing Machinery.
ACM ISBN 978-1-4503-7043-1/20/11...\$15.00
<https://doi.org/10.1145/3368089.3417933>

1 INTRODUCTION

Recently, with significant advances in cloud technologies and platforms, application modernization has become a hot topic. Many enterprises have established an imperative to move their monolithic applications to cloud. As of now, the primary focus has been on *lift-and-shift* approaches, where an entire application is containerized and moved as a single service to cloud. The limitations of this approach is that monolith applications, which have not been developed using cloud-native principles, cannot leverage cloud capabilities to their full potential. The applications remain difficult to scale, manage, and enhance. Although different approaches have been proposed in academic and industrial settings for refactoring applications, they have various limitations.

In industry settings, the most common approach that has been suggested is the bounded-context-based approach [6, 7, 12]. A bounded context takes the principle of domain-driven design (DDD) into consideration, where each domain corresponds to a service and a data model. The service owns this data, which represents an important property for a microservice—*i.e.*, a microservice is independent and decoupled from other microservices. But, in practice, refactoring efforts based on bounded contexts frequently fail because monolithic architectures have cross-functional object and data dependencies, and cannot be cleanly separated. No tools exist today that can align business functionality with the underlying object and data dependencies.

The approaches presented in academic research broadly fall under static or dynamic techniques. Approaches based on static analysis (*e.g.*, [2, 3, 5, 10, 11, 14, 16, 17]) capture static call relations among class objects and, therefore, face challenges in dealing with dynamic features of Java Enterprise Edition (JEE) applications, such as reflection, dynamic class loading, and context and dependency injections. Dynamic techniques (*e.g.*, [4, 8, 13]) capture runtime call relations and workload characteristics of an application; however the runtime traces do not capture alignment information about how classes and objects support different business functions which is a foremost concern in industry practices. Moreover, runtime techniques lack the complete structural information and programming constructs of the application that a static analysis provides.

We present Mono2Micro, an AI-based toolchain that assists users in migrating legacy JEE applications toward a microservice architecture. The tool addresses some key challenges in effectively partitioning the classes of an application to identify candidate microservices. First, Mono2Micro lets the application owner easily

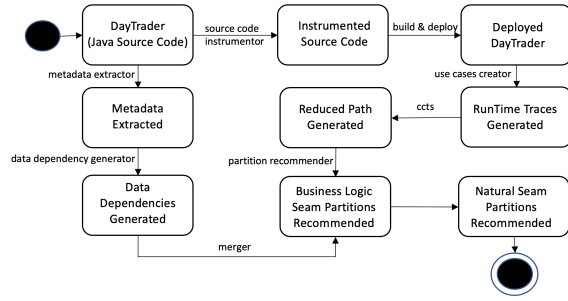


Figure 1: The Mono2Micro workflow for generating business-logic and natural-seams partitions.

define use cases and collect use-case-based execution traces on the application. These traces let runtime calls to be associated with business use cases. Mono2Micro also performs static analysis of the application to obtain structural information and data dependencies. These mappings are analyzed by an AI engine to create partitioning of application classes that considers business use case similarity and call dependencies. The resulting business-logic-seams-based partitions is an explainable grouping of classes that can be candidate microservices, utilities or business-context-related data entities. Second, Mono2Micro merges static data dependencies—captured as association relations between classes—with business-logic-seams-based partitions to recommend natural-seams-based partitions. The natural-seams-based partitions, unlike business-logic-seams-based partitions, do not cut across data dependencies and hence reduces the effort needed to transform the application to accommodate cross-partition data dependences.

In this paper, we describe the workflow and the pipeline of tools implemented in Mono2Micro. We illustrate the workflow and tools using a well-known open-source JEE application called DayTrader, which lets users log in and perform actions, such as buy and sell stocks and view market summary. DayTrader consists of 112 Java classes and 964 methods. We illustrate the business-logic and natural-seams partitions created for DayTrader based on a set of use cases and their associated traces.

Although our current focus is on JEE applications, the underlying approach of Mono2Micro is applicable more generally to web applications and even other types of applications where the core idea of leveraging use-case-based execution traces is applicable.

2 THE MONO2MICRO TOOLCHAIN

Figure 1 illustrates the workflow implemented in Mono2Micro, which is realized via a number of tools, such as the code instrumenter, the metadata extractor, and the AI-based partitions recommender. Next, we briefly describe and illustrate each tool.

2.1 Source-Code Instrumenter

The Mono2Micro workflow starts with code instrumentation. In this step, the source-code instrumenter scans all Java source files in the application to insert instrumentation statements in the form of `println("Entering ...")` and `println("Exiting...")` in all the class methods, including constructors. The instrumentation statements capture method calls and returns at runtime.

2.2 Metadata Extractor

In the next step, Mono2Micro’s metadata extractor parses the application classes to extract information, such as class names, class inheritance, their typed member attributes, constructors, methods with input arguments, local variables and return types, package dependencies, source files locations of the classes, and so on. For example, for class `TradeSLSBBean`, Mono2Micro extracts its member attributes (e.g., `queueConnectionFactory` and `tradeStreamerTopic`), constructor, and methods (e.g., `buy`, `sell`, and `getMarketSummary`), the classes it implements (`TradeSLSBRemote` and `TradeSLSBLocal`), its access modifier (i.e., `public`), and the source-file location.

2.3 Use-Cases Creator

Once the application is instrumented, it is built and deployed. Then, using Mono2Micro’s use-cases creator, we create use cases by navigating through the application’s user interface (UI) or by running functional test cases. Each use case is associated with a use-case label and has start and stop timestamps. For each such use case, we generate runtime traces, recording the method calls execute while that use case is exercised. If functional test cases are available for an application, they could be used as well for generating execution traces. The tests need not necessarily be UI test cases, but a test case must correspond to a well-defined application use case. We show an example of traces generated from DayTrader below. These traces are referred as *raw traces*.

```
t1,[32],Entering ... TradeAction::TradeAction [overloaded_#001]
t2,[32],Entering ... Log::doTrace
t3,[32],Entering ... Log::getTrace
t4,[32],Entering ... TradeConfig::getTrace
t4,[32],Exiting ... TradeConfig::getTrace
```

Each row of the trace captures a timestamp, a thread id, and an enter/exit label with a class and a method name. We use the thread id to piece together broken trace fragments. Mono2Micro provides a log processor that associates the raw traces with use-case labels based on their start and stop timestamps to generate traces tagged with use-case labels. The timestamps associated with a use-case label suggest runtime traces generated during the time period (stop-start). The traces generated by the added instrumentation can be easily distinguished, using their unique format, from any trace generated by application logging. The log processor generates the following tagged flow traces for the *login* use case.

```
t1, login, Root:<None> calls OrdersAlertFilter, ...
t2, login, OrdersAlertFilter calls TradeConfig, ...
t3, login, TradeConfig returns to OrdersAlertFilter, ...
t4, login, OrdersAlertFilter calls TradeAction, ...
t5, login, TradeAction calls Log, ...
```

Based on the call relationships between classes, we construct a directed graph (V, E) where V represents the classes and E represents the runtime call relations between classes. Figure 2 shows how Mono2Micro represents a runtime graph. Apart from a class-name, a node also provides additional information, such as its file path and the use cases it belongs to. Similarly, an edge provides information, such as its source and target nodes, and the total call volume between source and target nodes.

2.4 Calling-Context-Tree Creator

For each use case, Mono2Micro generates a set of tagged flow traces as shown in the previous section. To generate recommendations, we

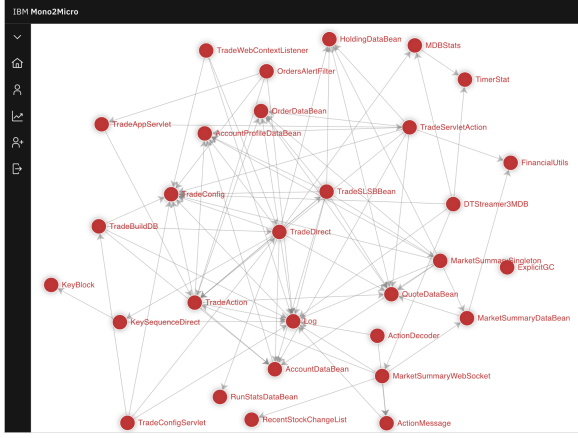


Figure 2: Runtime traces for DayTrader visualized as a directed graph, in which nodes represent classes and edges represent dynamic call relations between classes.

use the calling-context tree (CCT) [1]. We process the tagged flow traces to build a set of CCTs at the level of classes, with each tree rooted at an “entry point” class that is the first one to be invoked in response to a UI event. Thus, unlike the conventional CCT, where each node represents a method, our CCTs represent classes as nodes. This is because our recommended partitions do not have overlapping classes and they reflect distinct business functionalities within an application. Examples of such paths for a use case (*login*) is as follows:

```
login:[
OrdersAlertFilter,TradeConfig,
OrdersAlertFilter,TradeAction,Log,TradeConfig,
OrdersAlertFilter,TradeAction,TradeSLSBBean,Log,TradeConfig,
OrdersAlertFilter, ..., TradeSLSBBean,AccountProfileDataBean,
OrdersAlertFilter, ...,TradeSLSBBean,AccountDataBean,
]
```

2.5 AI Based Partitions Recommender

Mono2Micro’s partition recommender generates business-logic partitions using the reduced paths for each use case. To generate the partitions, the recommender applies temporo-spatial clustering on execution traces. For clustering, it uses the well-known hierarchical clustering algorithm [15] and the temporal relations between the classes. The hierarchical clustering algorithm takes two inputs: the target number of partitions k and the set of classes $C = \{c_1, c_2, \dots, c_n\}$ that occur in reduced paths. (The value k could be ascertained by the user in different ways, e.g., based in number of service entry points in an application.) Again, each class c_i is associated with m_i use cases, $B_i = \{b_{i1}, b_{i2}, \dots, b_{im_i}\}$, represented as user-provided use-case labels. For example, the *OrdersAlertFilter* class is associated with both *login* and *trade_sell* use-case labels. Initially, each class forms a partition by itself. During the clustering process, the similarity score $Sim_{i,j}$ for partition i and partition j is calculated using the following equation.

$$Sim_{i,j} = \frac{\sum_{m=0}^{n_i} \sum_{n=0}^{n_j} S(c_{im}, c_{jn})}{|C_i| |C_j|} \quad (1)$$

$S(c_{im}, c_{jn})$ is the similarity score of two classes c_{im} and c_{jn} belonging to the partition i and the partition j , respectively. $|C_i|$ and $|C_j|$ represents the total number of classes in the partitions i

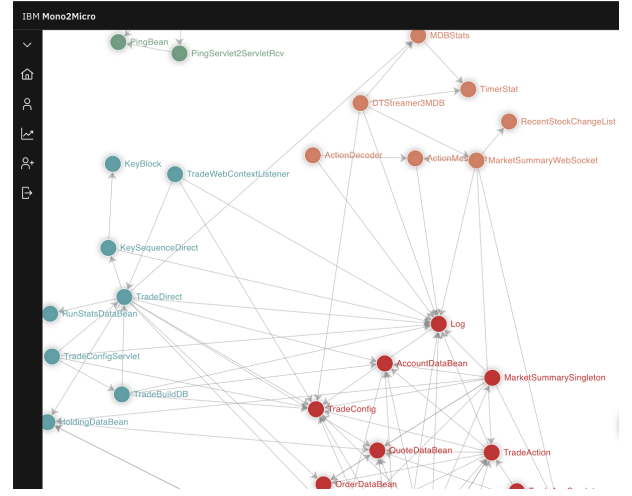


Figure 3: Business-logic-seams-based partitions generated for DayTrader; the colors represent different partitions.

and j , respectively. The similarity score $S(c_i, c_j)$ of a pair of classes c_i and c_j is calculated based on direct and indirect call relations (temporal relations). A *direct call relation* (DCR) exists between classes c_i and c_j if and only if edge (c_i, c_j) exists in the CCT; i.e., a method in c_i invokes a method in c_j in an execution trace. An example of a direct call relation exists between *OrdersAlertFilter* and *TradeConfig*. An *indirect call relation* exists between classes c_i and c_j if and only if there exists a path $(c_i, c_1, \dots, c_p, c_j)$, $p > 1$, in the CCT. An example of an indirect call relation exists between *OrdersAlertFilter* and *TradeSLSBBean*. Figure 3 shows the business-logic partitions generated for DayTrader.

2.6 Partitions Merger

To generate natural-seams-based partitions, first, Mono2Micro’s data-dependency generator obtains static data dependencies based on association relations between the classes: c_i has an association relation with class c_j if c_j is a member attribute of c_i . This suggests that there is an implicit reliance of c_i on the state of class c_j . Mono2Micro’s partition merger iteratively checks for each class c_i in a business-logic-seams-based partition whether c_i has a containment relation with class c_j in another partition. If such a relation exists, the partitions are merged. For example, in p_1 , the partition merger finds that *OrderDataBean* has an association relation with *HoldingDataBean* in p_2 . Thus, it merges the partitions to generate a natural-seams partition.

3 EMPIRICAL EVALUATION

We evaluate Mono2Micro on use-case-based traces for DayTrader provided by three different users. Because these use cases and traces are created by different users, they vary in terms of the number of use cases, use-case granularity, method and class coverage achieved, and the number of runtime calls that occur over all traces. Table 1 presents the details of the datasets. We classify the datasets based on their coverage percentage into three categories: small, medium and large. DayTrader (small) covers around 38% of total classes, whereas DayTrader (medium) and the DayTrader (large) cover around 66% of the classes. DayTrader (large) covers a higher percentage of method

Table 1: Three datasets for DayTrader with different use cases, coverage rates, and runtime calls.

Application	Cls	Met	# BC	Coverage		Calls
				Cls	Met	
DayTrader (small)	112	964	19	43 (38%)	333 (33%)	306,121
DayTrader (medium)	112	964	83	74 (66%)	428 (44%)	267,637
DayTrader (large)	112	964	4	74 (66%)	462 (47%)	1,000,839

Table 2: Sensitivity analysis on datasets for DayTrader with respect to three different cluster sizes: $k=5$, $k=7$, and $k=10$.

Application	k=5	k=7	k=10
DayTrader (small) (ICP)	0.32	0.32	0.43
DayTrader (small) (BCP)	1.71	1.12	0.99
DayTrader (small) (SM)	0.13	0.14	0.13
DayTrader (medium) (ICP)	0.37	0.37	0.37
DayTrader (medium) (BCP)	2.95	2.39	1.77
DayTrader (medium) (SM)	0.14	0.15	0.15
DayTrader (large) (ICP)	0.30	0.45	0.51
DayTrader (large) (BCP)	0.83	0.79	0.66
DayTrader (large) (SM)	0.09	0.10	0.07

calls than DayTrader (medium). Interestingly, for the larger dataset, the number of use cases is four, whereas for the medium one, the number of use cases is 83; the use cases for DayTrader (medium) are thus much finer-grained than the use cases for DayTrader (large).

3.1 Sensitivity Analysis

For the evaluation, we perform a sensitivity analysis, illustrated in Table 2, to show the robustness of our approach. Here, our assumption is that, based on a set of metrics, our algorithm can determine the best possible partitioning for a given application. For example, we evaluate the three datasets with different cluster sizes ($k=5$, $k=7$, and $k=10$) as inputs to the clustering algorithm. We measure the outcome on three metrics: (1) *inter-partition call percentage* (ICP) measures the percentage of runtime calls occurring between two partitions (thus, lower the ICP, better is the recommendation); (2) *business context purity* (BCP) measures the average entropy of business use cases per partition, i.e., $\frac{1}{N} \sum_{i=1}^N \frac{1}{|BC_i|} \log(\frac{1}{|BC_i|})$, where $|BC_i|$ represents the number of use cases for partition i (thus, lower the BCP, better is the recommendation); and (3) *structural modularity* (SM) measures the coupling from a structural perspective and calculates the trade-off between cohesion and coupling [9] (thus, higher the SM, better is the recommendation).

For DayTrader (small), we observe that $k=7$ provides the most optimal results with respect to ICP, BCP, and SM. With $k=10$, BCP improves but ICP and SM suffer. Thus, DayTrader (small) is most optimal with respect to 7 partitions. Increasing the number of partitions further does not affect the metrics. For DayTrader (medium), we find that results are most optimal with respect to $k=10$. This is because BCP significantly improves over smaller k values, with ICP and SM remaining constant. For DayTrader (large), we observe that, for $k=5$, our algorithm provides the most optimal results with respect to ICP, BCP, and SM. Increasing k to 7 causes BCP and SM to improve, but ICP suffers. With $k=10$, both ICP and SM suffer. This suggests that k is dependent on the number of use cases provided by the user. We also observe that increasing the value of k does not necessarily affect ICP and SM, which indicates additional partitions introduced by our algorithm may not have classes with

Table 3: Partitions computed for DayTrader for $k=7$.

p_1	p_2	p_3
OrdersAlertFilter	TradeWebContextListener	MDBStats
TradeAppServlet	TradeConfigServlet	TimerStat
TradeServletAction	TradeBuildDB	DTStreamer3MDB
FinancialUtils	JSFLoginFilter	ActionDecoder
Log	KeySequenceDirect	ActionMessage
TradeConfig	TradeDirect	MarketSummaryWebSocket
MarketSummaryDataBean	KeyBlock	RecentStockChangeList
QuoteDataBean	RunStatsDataBean	
AccountProfileDataBean	HoldingDataBean	
...		
TradeSLSBBean		

core business functionalities. We next discuss the quality of the computed partitions.

3.2 Partition Analysis

Table 3 shows three partitions from DayTrader (small) based on $k=7$ that reflect core business functionalities. Classes associated with partition p_1 include functionalities such as buying and selling stocks, classes associated with partition p_2 include functionalities related to database create and populate, and classes associated with partition p_3 include functionalities such as messaging. The functionalities are indicative of the use cases created by users and reflect independent business modules of DayTrader. For verification, we showed the partitions to experienced IT architects who are familiar with DayTrader and they positively affirmed the recommendations. For DayTrader (medium) and DayTrader (large), we obtained similar partitions with core business functionalities based on $k=10$ and $k=5$, respectively.

4 CONCLUSION

In this paper, we describe and demonstrate the Mono2Micro toolchain as applied to an enterprise Java application to recommend partitions. We also conducted analysis of the toolchain under varied parameter settings. Our initial experience with Mono2Micro has yielded promising results. In future work, we will conduct further evaluations of Mono2Micro on real-world client datasets and refine the techniques implemented in the Mono2Micro toolchain. We are also developing techniques and tools for supporting the transformation steps that come after partitioning, such as building and deploying each partition as a set of services.

The Mono2Micro tool is available at: <https://ibm.biz/Mono2Micro>. The demo dataset is available at: <https://tinyurl.com/yy7cmo2>.

ACKNOWLEDGMENTS

We would like to thank our partners at IBM Cloud and Cognitive Software (Danny Mace, Melissa Modjeski, Garth Tschetter, Labros Kisouras, and Vikram Gulati) and the Hybrid Cloud development team (Sasa Matijevic, Leonard Theivendra, Anuradha Ramamoorthy, Laven Sathiyathan, Lavena Chan, Tiaoyu Wang, and Benson Ning), without whom Mono2Micro would not have been realized. We are grateful to Shivali Agarwal, Amith Singhee, Srikanth Tamilselvan, Yasuharu Katsuno, Fumiko Satoh, Nicholas Fuller, and Ruchir Puri for useful suggestions and feedback.

REFERENCES

- [1] Glenn Ammons, Thomas Ball, and James R. Larus. 1997. Exploiting Hardware Performance Counters with Flow and Context Sensitive Profiling. In *Proceedings of the ACM SIGPLAN 1997 Conference on Programming Language Design and Implementation*. Association for Computing Machinery, 85–96. <https://doi.org/10.1145/258915.258924>
- [2] Luiz Carvalho, Alessandro Garcia, Wesley K. G. Assunção, Rodrigo Bonifácio, Leonardo P. Tizzei, and Thelma Elita Colanzi. 2019. Extraction of Configurable and Reusable Microservices from Legacy Systems: An Exploratory Study. In *Proceedings of the 23rd International Systems and Software Product Line Conference - Volume A*. 26–31. <https://doi.org/10.1145/3336294.3336319>
- [3] Rui Chen, Shanshan Li, and Zheng Li. 2017. From Monolith to Microservices: A Dataflow-Driven Approach. In *Proceedings of 24th Asia-Pacific Software Engineering Conference*. 466–475.
- [4] Adambarage Anuruddha Chathuranga De Alwis, Alistair Barros, Artem Polyvyanyy, and Colin Fidge. 2018. Function-Splitting Heuristics for Discovery of Microservices in Enterprise Systems. In *International Conference on Service-Oriented Computing*. Springer, Hangzhou, 37–53.
- [5] Daniel Escobar, Diana Cardenas, Rolando Amarillo, Eddie Castro, Kelly Garcés, Carlos Parra, and Rubby Casallas. 2016. Towards the understanding and evolution of monolithic applications as microservices. In *Proceedings of Latin American Computing (CLEI)*. IEEE, Chile, 1–11.
- [6] Martin Fowler. 2014. BoundedContext. <https://martinfowler.com/bliki/BoundedContext.html>
- [7] Martin Fowler. 2019. *Microservices Guide*. Retrieved May 15, 2020 from <https://martinfowler.com/microservices/>
- [8] Wuxia Jin, Ting Liu, Yuanfang Cai, Rick Kazman, Ran Mo, and Qinghua Zheng. 2019. Service Candidate Identification from Monolithic Systems based on Execution Traces. *IEEE Transactions on Software Engineering* (Apr 2019), 1–21.
- [9] Wuxia Jin, Ting Liu, Yuanfang Cai, Rick Kazman, Ran Mo, and Qinghua Zheng. 2019. Service candidate identification from monolithic systems based on execution traces. *IEEE Transactions on Software Engineering* (2019).
- [10] Alessandra Levcovitz, Ricardo Terra, and Marco Tulio Valente. 2016. Towards a Technique for Extracting Microservices from Monolithic Enterprise Systems. *CoRR* (2016), 1–8. <http://arxiv.org/abs/1605.03175>
- [11] Genc Mazlami, Jürgen Cito, and Philipp Leitner. 2017. Extraction of Microservices from Monolithic Software Architectures. In *Proceedings of International Conference on Web Services*. IEEE, Honolulu, 524–531.
- [12] Sam Newman. 2015. *Building Microservices* (1st ed.). O'Reilly Media, Inc.
- [13] Chiragkumar Patel, Abdelwahab Hamou-Lhadj, and Juergen Rilling. 2009. Software Clustering Using Dynamic Analysis and Static Dependencies. In *Proceedings of the 2009 European Conference on Software Maintenance and Reengineering*. IEEE Computer Society, 27–36. <https://doi.org/10.1109/CSMR.2009.62>
- [14] Zhongshan Ren, Wei Wang, Guoquan Wu, Chushu Gao, Wei Chen, Jun Wei, and Tao Huang. 2018. Migrating Web Applications from Monolithic Structure to Microservices Architecture. In *Proceedings of the Tenth Asia-Pacific Symposium on Internetware*. ACM, Beijing, 1–10.
- [15] Robin Sibson. 1973. SLINK: An optimally efficient algorithm for the single-link cluster method. *Comput. J.* 16, 1 (jan 1973), 30–34.
- [16] Davide Taibi and Kari Systä. 2019. From monolithic systems to microservices: a decomposition framework based on process mining. In *9th International Conference on Cloud Computing and Services Science (CLOSER)*. Greece.
- [17] Junfeng Zhao, Jiantao Zhou, Hongji Yang, and Guoping Liu. 2015. An Orthogonal Approach to Reusable Component Discovery in Cloud Migration. *China Communications* 12, 5 (2015), 134–151.